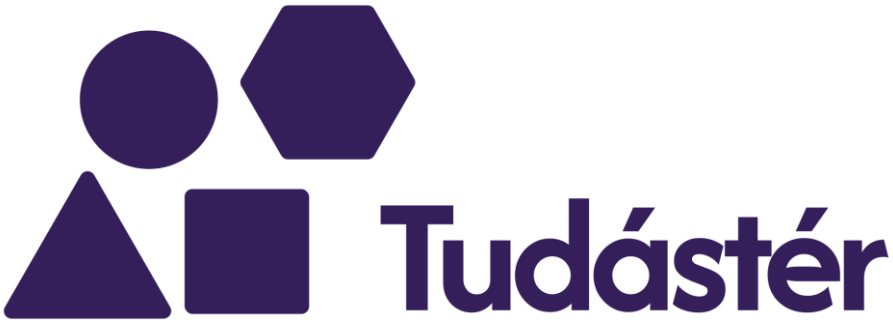
Tudástér

Algoritmus specifikáció



Készítették: Keserű Fruzsina 13.D

Barta Hunor Attila 13.D

Tartalomjegyzék

1 AI-alapú PDF-konvertálás 3
1.1 PDF feldolgozása (Import) 3
1.2 AI hívás (Gemini API) 4
1.3 Leképezés és ellenőrzés 5
1.4 Piszkozatok kezelése (helyi tárolás) 5
1.5 Piszkozat szerkesztése 6
2 Regisztráció és bejelentkezés 8
2.1. Regisztráció (Register) 8
2.2. Bejelentkezés (Login) 9
3 Tesztek kitöltését támogató algoritmusok 11
3.1. Folyamatban lévő teszt rögzítése 11
3.2. Teszt befejezése 13

1 AI-alapú PDF-konvertálás

1.1 PDF feldolgozása (Import)

Alapelvek:

- Az AI eredménye nem kerül automatikusan az adatbázisba, előbb helyi piszkozatként mentődik, és az admin minden feladatot ellenőriz és jóváhagy.
- A program az AI válaszát kódból is ellenőrzi (sorszámok, pontösszeg, forrásszövegek).
- Az API-kulcs nem a forráskódban, hanem a felhasználói titkok (user secrets) között tárolódik.

Cél: Egy érettségi feladatlap PDF-jének automatikus feldolgozása: a feladatok felismerése, majd mentése helyi piszkozatként.

UI Interakció:

- Az Importálás oldalon az admin a PDF KIVÁLASZTÁSA gombbal kiválaszt egy .pdf fájlt, majd a PDF FELDOLGOZÁSA gombbal elindítja a feldolgozást.
- Feldolgozás közben a gombok le vannak tiltva, és a „Feldolgozás folyamatban...“ felirat látszik.
- Hiba esetén (hiányzó API-kulcs, hálózati hiba, elfogyott kvóta, értelmezhetetlen válasz) hibaüzenet jelenik meg a hiba okával.
- Siker esetén megjelenik a felismert feladatok száma, a feladatsor címe, az esetleges figyelmeztetések és a MEGNYITÁS A SZERKESZTŐBEN gomb. A piszkozat a Mentett piszkozatok listában is megjelenik.

Bemenet: PDF fájl elérési útja; beállítások: Gemini:ApiKey, Gemini:Model; a témakörök listája.

Kimenet: helyi piszkozat (DraftExam); a nyers AI-válasz hibakereséshez (exam.json az ideiglenes mappában).

UI kimenet: sikerüzenet figyelmeztetésekkel és a szerkesztő megnyitása, vagy hibaüzenet.

Eljárás:

1. Ellenőrzi, hogy a kiválasztott fájl létezik, és .pdf kiterjesztésű.
2. PdfPig segítségével oldalanként kinyeri a PDF szövegét. Erre csak a forrásszövegek későbbi ellenőrzéséhez van szükség.
3. Elküldi a PDF-et és az indításkor betöltött témaköröket (azonosító, név, tantárgy) a Gemini API-nak (lásd 1.2). Az API-kulcsot a user secretsből olvassa, ha nincs kulcs → hibaüzenet.
4. A kapott választ leképezi és ellenőrzi (lásd 1.3), az eredmény egy DraftExam objektum.
5. Elmenti a piszkozatot és az eredeti PDF másolatát (lásd 1.4), majd frissíti a piszkozatok listáját.
6. Ha bármelyik lépésnél hiba lép fel, a folyamat megszakad, semmit sem ment, a gombok pedig újra aktívak lesznek.

1.2 AI hívás (Gemini API)

Cél: A PDF tartalmának átalakítása strukturált, géppel feldolgozható JSON adattá.

Bemenet: PDF bájtok (Base64), utasítás (prompt), témakörök listája, a kért JSON-séma.

Kimenet: AiExamDto: a vizsga adatai (tantárgy, szint, év, összpontszám), források és feladatok.

Eljárás:

1. Összeállítja a kérést: utasítás + témakörök listája + a PDF + a kért JSON-séma (responseSchema), 0 temperature, hogy a válasz ismételhető legyen.
2. Az utasítás lényege: a címoldalt, a fejléceket és a pontozó táblázatot hagyja ki; a szövegeket szó szerint másolja; a forrásokat, a részkérdéseket (a, b, c), a válaszlehetőségeket és a táblázat celláit külön adja meg; ábránál csak jelezze, hogy tartozik hozzá; bizonytalanságát jelezze; témakört csak a megadott listából javasolhat.
3. A kérést HTTPS POST-tal küldi el, az API-kulcs a fejlécben utazik. 429-es vagy 5xx hibánál legfeljebb háromszor újra próbálja (5, 15, 45 mp), más hibánál kivételt dob.
4. A választ JSON-ként értelmezi. Ha üres vagy levágódott, hibát jelez.

1.3 Leképezés és ellenőrzés

Cél: Az AI válaszának átalakítása a Tudástér adatmodelljére, és megbízhatóságának ellenőrzése kódból.

Bemenet: AiExamDto, a PDF kinyert szövege, a témakörök listája.

Kimenet: DraftExam, amely feladatonként és feladatsor szinten figyelmeztetéseket tartalmaz.

Eljárás:

1. Átveszi a vizsga adatait (tantárgy, év, szint) és a forrásszövegeket.
2. Ellenőrzi, hogy a feladatok sorszáma folytonos-e, a feladatpontok összege egyezik-e az összpontszámmal, a részpontok összege egyezik-e a feladat pontjával, és hogy a feleletválasztós feladatnak van-e legalább két válaszlehetősége.
3. Ellenőrzi, hogy a forrásszöveg szó szerint szerepel-e a PDF szövegében (50 karakteres minták legalább 80%-ának egyeznie kell, a szóközöket, sortöréseket és kötőjeleket figyelmen kívül hagyva).
4. Összeállítja a kérdést a részkérdésekkel, a táblázatot oszlopokra és cellákra bontja, a témakört csak létező azonosítóval fogadja el.
5. Eltérés esetén figyelmeztetést ad, és ellenőrzésre jelöli azt a feladatot, amelyhez figyelmeztetés, ábra vagy táblázat tartozik.

1.4 Piszkozatok kezelése (helyi tárolás)

Cél: A feldolgozott feladatsor biztonságos tárolása az adatbázisba töltés előtt, hogy az admin bármikor folytathassa a munkát, akár az alkalmazás újraindítása után is.

UI Interakció:

1. Az Importálás oldal alján a Mentett piszkozatok lista látható. Minden piszkozat egy kártya: a feladatsor címe (év, tantárgy, szint), a feladatok száma, a jóváhagyott feladatok száma és az utolsó módosítás ideje.
2. A MEGNYITÁS gomb megnyitja a szerkesztőt, a lista a szerkesztő bezárása után frissül.
3. A TÖRLÉS gomb megerősítő kérdést jelenít meg („A hozzá tartozó képek is törlődnek.”). Igen esetén a piszkozat törlődik.

Bemenet: DraftExam objektum, azonosító.

Kimenet: fájlok a helyi tárolóban, a piszkozatok összesítő listája.

UI kimenet: frissített lista.

Eljárás:

1. Minden piszkozat saját mappát kap:
   %AppData%\\TudasterAdmin\\drafts\\<azonosító>\\.
2. 2. A mappa tartalma: draft.json (a feladatsor szerkeszthető állapota), source.pdf (az eredeti feladatlap), answerkey.pdf (a megoldókulcs, ha betöltötték), images\\ (a hozzárendelt képek).
3. Mentéskor a program először ideiglenes fájlba ír, majd azt cseréli a draft.json-ra, így egy hiba nem rongálja meg a meglévő mentést.
4. A lista felépítésekor a program beolvassa a mappákat, a sérült piszkozatokat kihagyja, a többit módosítás ideje szerint rendezi (legújabb elöl).
5. Törléskor a piszkozat teljes mappája törlődik.

1.5 Piszkozat szerkesztése

Cél: A felismert feladatok ellenőrzése, javítása, kiegészítése képpel, majd jóváhagyása.

UI Interakció:

- Bal oldalon a feladatok listája látható jelekkel: „✓“ jóváhagyott, „⚠“ hiányos, „○“ még nem ellenőrzött. Egy feladatra kattintva jobb oldalon szerkeszthető a típus, pont, témakör, kérdés, válaszlehetőségek, helyes válaszok, megoldás menete és a forrásszöveg.
- A KÉP HOZZÁRENDELÉSE gombbal képet lehet a forráshoz csatolni (forrásonként egyet).
- A táblázat szövegként szerkeszthető: az első sor a fejléc, a cellákat | jel választja el, a ?-lel kezdődő cellát a diák tölti ki.
- A „Jóváhagyva“ jelölőnégyzet jelzi, hogy az admin ellenőrizte a feladatot. Feladatváltáskor és bezáráskor a szerkesztő automatikusan ment.
- Az ELŐZŐ és KÖVETKEZŐ gombokkal vagy a listára kattintva lép a feladatok között. Váltáskor és bezáráskor a szerkesztő automatikusan ment, a MENTÉS gomb azonnal ment és kiírja a mentés idejét.

Bemenet: DraftExam, témakörök listája, tantárgyak listája.

Kimenet: módosított DraftExam a helyi tárolóban.

UI kimenet: frissített feladatlista jelekkel, mentési visszajelzés.

Eljárás:

1. Megnyitáskor betölti a piszkozatot, feladatváltáskor visszaírja az űrlapot a modellbe és elmenti.
2. A témakörlista a tantárgy szerint szűkül. A „Hiányzik“ jelzés a kérdés, a pont, a típus és a témakör kitöltetlenségét mutatja.
3. Kép hozzárendelésekor a kép a piszkozat images mappájába másolódik. Ha a feladathoz még nincs forrás, egy üres szövegű forrás jön létre, mert az adatbázisban a forrás szövege kötelező.
4. Az adatbázisba töltés feltétele: a feladatsor adatai kitöltöttek, és minden feladat jóváhagyott.

2 Regisztráció és bejelentkezés

2.1. Regisztráció (Register)

Cél: A felhasználó regisztrációjának biztosítása, a megadott adatok ellenőrzése, majd a felhasználói fiók létrehozása a Supabase Auth rendszerében és a hozzá tartozó profil rögzítése az adatbázisban.

UI Interakció:

1. A felhasználó megnyitja a regisztrációs felületet.
2. Megadja az alábbi mezőket: email, felhasználónév jelszó, jelszó megerősítése, szerepkör.
3. Az űrlapot elküldi.
4. Hibák:
    - Üres mező -> piros hiba figyelmeztető üzenet
    - Már foglalt felhasználónév -> "Ez a felhasználónév már foglalt." üzenet
    - Nem megfelelő jelszó -> "A jelszó nem felel meg a követelményeknek." üzenet
    - Nem egyezik a jelszó és a jelszó megerősítése mező -> "A jelszavak nem egyeznek." üzenet

Bemenet: email, jelszó, jelszó megerősítése, felhasználónév, szerep

Kimenet: user objektum, message

UI Kimenet: Siker üzenet és jelzi, hogy email-es megerősítésre vár a regisztráció.

Email Interakció: A regisztráció megerősítésre kerül és átirányítjuk a főoldalra bejelentkezve.

Eljárás:

1. Ellenőrzi, hogy minden mező jelen van-e és helyes.
2. Ellenőrzi, hogy az email és a felhasználónév foglalt-e már az adatbázisban.
3. Meghívja a supabase.auth.signUp() Supabase Auth függvényt, ami menti a felhasználót az auth sémába, itt automatikusan biztonságosan tárolódik a jelszó.
4. A handle_new_user Supabase PostgreSql RPC függvény az auth sémába érkező adatok alapján a publikus sémában lévő profilok táblába is menti az adatokat.
5. A felhasználónak a megadott emailre regisztrációt megerősítő emailt küld.

6. Regisztráció után a Supabase Auth visszaadja a bejelentkezett állapothoz szükséges adatokat (user, session: access token, refresh token, expires_at) és automaitkusan bejelekeztetjük az új felhasználót.

2.2. Bejelentkezés (Login)

Cél: A már regisztrált felhasználó azonosítása a Supabase Auth rendszerén keresztül, a bejelentkezési munkamenet létrehozása, valamint a védett funkciókhoz szükséges hozzáférés biztosítása.

UI Interakció:

1. A felhasználó megnyitja a bejelentkezési oldalt.
2. Megadja az e-mail-címét és a jelszavát.
3. Majd a Bejelentkezés gombra kattint.
4. Hiba esetén (üres mező vagy helytelen adat) → "Hibás email cím vagy jelszó." hiba üzenet jelenik meg
5. Sikeres bejelentkezés esetén átirányítjuk főoldalra.

Bemenet: email, password

Kimenet: user, session (access token, refresh token, expires_at) objektum vagy error objektum

UI Kimenet: Főoldalra átirányítás vagy hiba üzenet megjelenítése

Eljárás:

1. Az űrlap elküldésekor a frontend megakadályozza az oldal alapértelmezett újra töltését, törli az előző hibaüzenetet, és betöltési állapotba kapcsol.
2. A frontend meghívja a supabase.auth.signInWithPassword() Supabase Auth függvényt, átadva az e-mail-címet és a jelszót.
3. A Supabase Auth ellenőrzi a megadott hitelesítési adatokat.
4. Hibás adatok esetén a függvény error objektumot ad vissza. A frontend megjeleníti a hibaüzenetet, majd megszünteti a betöltési állapotot.
5. Sikeres hitelesítés esetén a Supabase létrehozza a munkamenetet (session), amely tartalmazza többek között az access_token és refresh_token értékeket, valamint a felhasználó adatait.

6. Az AuthContext.jsx fájlban regisztrált supabase.auth.onAuthStateChange() Supabase Auth eseményfigyelő érzékeli a bejelentkezést, és frissíti a session állapotát. A user értékét a rendszer ebből a munkamenetből határozza meg minden műveletnél.
7. A frontend átirányítja a felhasználót a főoldalra.

3 Tesztek kitöltését támogató algoritmusok

3.1. Folyamatban lévő teszt rögzítése

Cél: Új tesztkitöltés indítása, a kiválasztott feladatok és a felhasználó összekapcsolása, valamint a teszt adatbázisban történő rögzítése folyamatban állapottal.

UI Interakció:

1. A bejelentkezett felhasználó kiválasztja milyen tesztet szeretne kitölteni. Megadja melyik tantárgyból, közép vagy emelt szinten és hogy adott év feladatsorát vagy adott témakörhöz kapcsolódó feladatokat akar kitölteni.
2. Elindítja a teszt kitöltését.

Bemenet: tantargy_id, szint, ev vagy temakor_id

Kimenet: feladat objektumok

Bemenet:

- tantargy_id – a kiválasztott tantárgy azonosítója a feladat
- feladat_idk – a tesztbe kerülő feladatok azonosítóinak tömbje a feladat objektumokból összeállítva.
- Felhasználói azonosító (userId) – a backend által ellenőrzött JWT-tokenből származik majd az API hívás header részéből nyerhető ki.

Kimenet:

Sikeres tesztindítás esetén a backend HTTP 201 státuszkóddal a következő adatokat adja vissza:

- teszt_id – a létrehozott teszt azonosítója.
- statusz – a teszt aktuális állapota (folyamatban).
- feladat_idk – a teszthez rendelt feladatok azonosítói.

Hibás bemenet vagy sikertelen adatbázis-művelet esetén a backend hibaválaszt ad vissza.

UI kimenet:

- Sikeres indítás esetén a felület megkapja az új teszt azonosítóját és a teszthez rendelt feladatok listáját, amelyek alapján megkezdhető a teszt kitöltése.

- Hibás adatok esetén a felület a backend által visszaadott hibaüzenetet jelenítheti meg. Sikertelen adatbázis-művelet esetén a teszt létrehozása nem tekinthető sikeresnek.

Eljárás:

1. A felhasználó elindítja a tesztet a felületen.
2. A frontend HTTP GET-kérést küld a backend /taskroutes/chosentasks végpontjára az elvárt feladat tulajdonságokkal: tantárgy azonosítója, szint, melyik év feladatsora vagy melyik témakörhöz tartozzon témakör azonosító alapján
3. A frontend HTTP POST-kérést küld a backend /taskroutes/starttest végpontjára a tantárgy azonosítójával, és az előbb visszakapott feladatok azonosítóival.
4. A backend requireAuth middleware-e ellenőrzi a felhasználó hitelesítését. A felhasználó azonosítóját az ellenőrzött tokenből veszi át, nem a kliens által küldött adatokból.
5. A /taskroutes/starttest végpont ellenőrzi a bemeneteket:
    - A tantárgy azonosítójának pozitív egész számnak kell lennie;
    - A feladatok listája nem lehet üres, és csak pozitív egész számokat tartalmazhat
6. A backend meghívja a startTest() függvényt a testCorrector.js-ből, átadva a hitelesített felhasználó azonosítóját, a tantárgy azonosítóját, a feladatok listáját és a teszt nevét.
7. A startTest() kiszűri a duplikált feladatazonosítókat, miközben megőrzi azok sorrendjét.
8. Az adatbázis-műveletek egyetlen tranzakción belül futnak. A rendszer ellenőrzi, hogy minden megadott feladat létezik-e, és a kiválasztott tantárgyhoz tartozik-e. Ha az ellenőrzés sikertelen, a tranzakció nem rögzíti a tesztet.
9. Ha minden feladat megfelelő, a backend új rekordot hoz létre a tesztek táblában. A rekord tartalmazza a felhasználó azonosítóját, a tantárgy azonosítóját, a teszt nevét, az indítás időpontját és a folyamatban állapotot.
10. A rendszer a teszt_feladatok táblában összekapcsolja az új tesztet a hozzá tartozó feladatokkal, a kiválasztott sorrendet megtartva.
11. Ha mindkét adatbázis-művelet sikeres, a tranzakció véglegesül. Hiba esetén a tranzakció visszagörgetésre kerül, így nem marad félkész tesztrekord.
12. A backend visszaadja az új teszt azonosítóját, állapotát és a feladatok azonosítóit a frontend számára.
13. A teszt elindul a frontenden.

3.2. Teszt befejezése

Cél: A kitöltött teszt válaszainak fogadása, ellenőrzése és javítása, az eredmények adatbázisban történő mentése, majd a felhasználó tájékoztatása a teszt eredményéről.

UI Interakció:

1. A felhasználó befejezi a teszt kitöltését, majd a beadás gombra kattint.
2. A rendszer elküldi a teszt azonosítóját, a válaszokat és a kitöltés időtartamát a backendnek, küldés alapján azonosítható a felhasználó is.

Bemenet:

- teszt_id – a beadott teszt azonosítója, amit a teszt indításakor kaptunk válaszként
- valaszok – a feladatok azonosítóit, típusait és a felhasználó válaszait tartalmazó tömb.
- ido – a teszt kitöltésével eltelt idő.
- Felhasználói azonosító (userId) – a backend által ellenőrzött JWT-tokenből származik majd az API hívás header részéből nyerhető ki.

Kimenet:

- A teszt azonosítója.
- A javított feladatok eredményei.
- Az elért pontszámok feladatonként.
- A kitöltési idő.
- A teszt végleges állapota.

Hiba esetén a backend hibakódot és hibaüzenetet ad vissza.

UI Kimenet:

1. A feldolgozás ideje alatt a felület jelezi, hogy a javítás folyamatban van.
2. Sikeres beadás után a felhasználó megtekintheti a teszt eredményét, a pontszámot és a feladatok javítását.
3. Hibás beadás vagy szerverhiba esetén a felület hibaüzenetet jelenít meg.

Eljárás:

1. A felhasználó beadja a tesztet. A frontend POST-kérést küld a backend /correctionroutes/incoming végpontjára, a teszt azonosítójával, a válaszokkal és a kitöltési idővel.
2. A backend ellenőrzi a beérkezett adatok szerkezetét és értékeit. Vizsgálja többek között a tesztazonosítót, a válaszok számát, a feladattípusokat, a duplikált feladatazonosítókat és az időtartamot.
3. A backend requireAuth middleware-e ellenőrzi a felhasználó hitelesítését. A felhasználó azonosítóját az ellenőrzött tokenből veszi át, nem a kliens által küldött adatokból.
4. A testCorrector.js-ből meghívja a receiveSubmission() függvényt a hitelesített felhasználó, teszt azonosító, válaszok és a másodperccé alakított adatok segítségével.
5. A parositTesztet() függvény ellenőrzi, hogy a teszt létezik-e, a bejelentkezett felhasználóhoz tartozik-e, és még folyamatban állapotú-e. Azt is ellenőrzi, hogy a beküldött feladatok valóban a teszthez tartoznak-e.
6. Ha az ellenőrzés sikertelen, a rendszer megszakítja a folyamatot, és hibaválaszt küld a frontendnek.
7. A feldolgozBeadast() függvény a teszt állapotát folyamatban értékről javitas értékre állítja. Ezzel megakadályozza, hogy ugyanazt a tesztet párhuzamosan vagy ismételten beadják.
8. Kiszámoljuk a kitöltési idejét a tesztnek.
9. A rendszer betölti a teszthez tartozó feladatok helyes megoldásait, típusait és pontértékeit az adatbázisból.
10. Egy adatbázis-tranzakcióban elmenti a javítás eredményeit, a pontszámokat és a kitöltési időt, majd a teszt állapotát befejezett értékre állítja.
11. Válaszként a teszt azonosítóját és státuszát küldi vissza a frontendnek.
12. A frontend a felhasználó, a teszt azonosítója és státusza alapján lekéri a megoldásokat és megjeleníti azokat.